

Neural Wani

Toward Accelerating the Automated Theorem Prover *wani*
for Dependent Type Theory

Nanako Miyagawa Hinari Daido Daisuke Bekki



Ochanomizu University

NALOMA 2026 | August 6, 2026

1

Motivation / Background

Understanding Natural Language Semantics

Natural Language Inference (NLI) pipeline based on formal linguistics [Tomita et al. 2025]

Outputs **the results of type checking and proof search (proof diagrams)**.

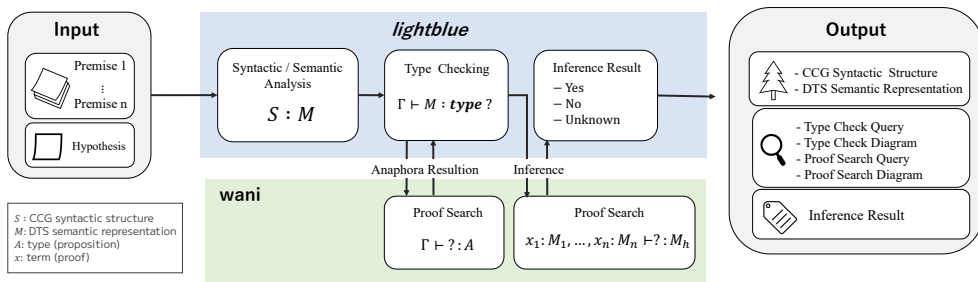


Figure 1 of [Tomita et al. 2025]

wani [Daido and Bekki 2020]

Automated theorem prover based on **Dependent Type Theory (DTT)** [Martin-Löf 1984]. A DTT judgment:

$$\Gamma \vdash M : A$$

context term type
(premises) (proof) (proposition)

Curry–Howard: a proof M exists for conclusion A given premises Γ .

Dependent Type Semantics (DTS) [Bekki 2014; Bekki and Mineshima 2017]

The semantic theory built on DTT. Π - and Σ -types let consequent types depend on antecedents (anaphora), and as a **proof-theoretic** semantics, inference runs **purely through proof search** — with no external models.

Proof Search in *wani*

A judgment whose **proof term is unknown** is a **proof search query**:



A **proof term** is found $\rightarrow$ the premises entail the hypothesis.

Two Directions of Inference



To resolve these queries, *wani* **combines two directions**:

Forward inference ↓

top-down derivation
from premises

Backward inference ↑

bottom-up construction
of subgoals

Limits of Proof Search in *wani*

wani restricts the DTT framework to a **fragment** and employs depth-first search with **predefined depth and time limits**.

Reason for these constraints

DTT — the higher-order foundation of DTS — possesses greater expressive power than first-order logic. The price of that power is **undecidability**: proof discovery cannot be guaranteed, so the search must be **bounded**.

- Within these bounds, under depth-first search, an unsuitable rule applied **early** makes computation time **grow exponentially**.

2

Proposed Method

Neural Wani

Proposal of *Neural Wani* [Miyagawa et al. 2025]

Integrating a neural network into *wani*

Assisting the proof search query $\Gamma \vdash ? : A$ using a **neural model**.

wani

- High computational cost during the search process
- **Logically sound** due to proof-theoretic inference



Neural Model

- **Returns results rapidly** without searching
- Predicts probabilistically

Combining the strengths of both approaches to enable efficient proof search.

Integrating the Neural Network

Since *wani* relies on depth-first search, exploring unproductive branches in the early stages leads to **high computational latency**.

→ **Prioritizing search paths based on probabilities predicted by the neural network.**

No rule is excluded — the neural network only decides which to try first.

Where Does the Model Intervene?

Learning-based provers differ by **which component the model informs**:

- **Premise selection** — score which facts help a proof [Wang et al. 2017; Kogkalidis et al. 2024].
- **Differentiable inference engine** — learn symbol similarities end-to-end [Rocktäschel & Riedel 2017].

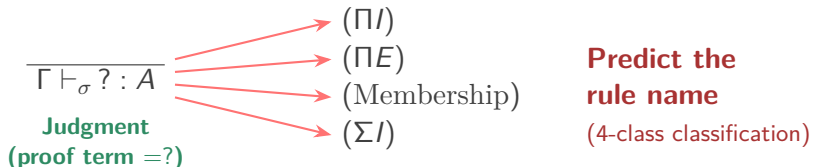
Neural Wani (ours): a third locus

The **order in which inference rules are expanded** during backward search.

- Premises are supplied upstream $\Rightarrow$ premise selection is not the bottleneck.
- *wani*'s symbolic engine is left **unchanged** (not a differentiable approximation) $\Rightarrow$ **soundness intact**.

Predicting Rule Names from Judgments

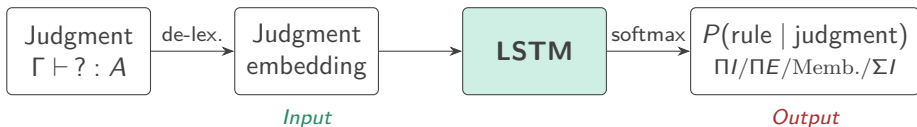
Neural Wani works inside *wani*'s **backward inference**, and is queried at **every node** of the search tree.



Applying a rule generates new subgoals — again judgments without proof terms.

Neural Model Architecture

Following our previous work [Miyagawa et al. 2025], we employ an **LSTM** [Hochreiter and Schmidhuber 1997] for rule prediction.



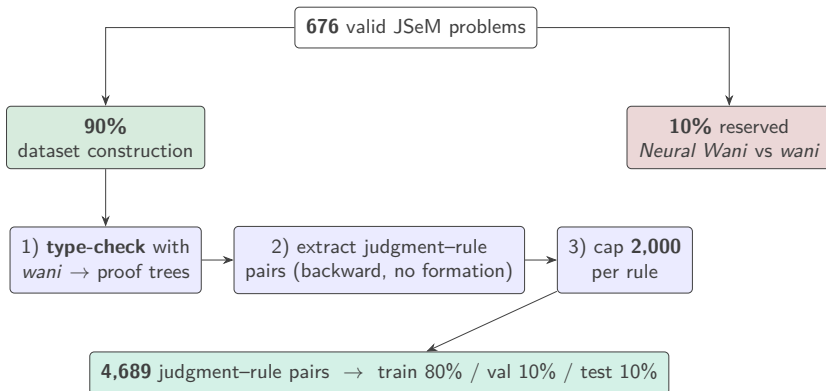
3

Evaluation Experiments

Neural Wani

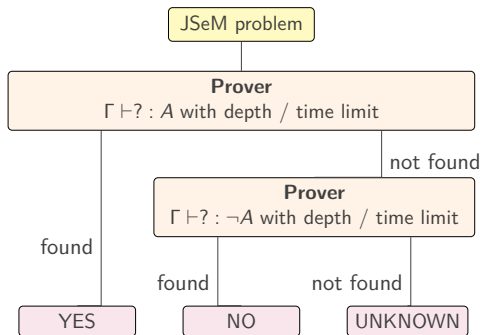
Training and Evaluation Dataset

NLI problems from **eight sections of JSeM** [Kawazoe et al. 2015a, 2015b], a **Japanese** semantic test suite, parsed via *lightblue* [Bekki and Kawazoe 2016] and composed in DTS.



Evaluation Methodology for *Neural Wani*

Integrated into baseline *wani*; evaluated on **50 held-out NLI problems** (backward rules **prioritized by the model**, no top- k pruning).



wani: intuitionistic mode, depth limit **9**, time limits **30 / 60 / 90 s**. **YES**: entail **NO**: contradict
UNK: neither within limit.

4

Results & Discussion

Neural Wani

Overall Classification Performance

Overall	Prec	Rec	F1	Supp
micro	0.821	0.821	0.821	470
macro	0.834	0.608	0.591	470
w-avg	0.834	0.821	0.821	470

Per label	Prec	Rec	F1	Supp
(ΠI)	0.582	0.768	0.662	69
(ΠE)	0.866	0.710	0.780	200
(Memb.)	0.888	0.955	0.920	200
(ΣI)	1.000	0.000	0.000	1

What these numbers tell us

- 0.821 against 0.982 *with* proof terms [Miyagawa et al. 2025]: the term is the **strongest cue**.
- (Memb.) and (ΠE) dominate backward search, and are predicted **most accurately** — where the search cost is.
- macro-F1 is held down by (ΣI) : **one** test instance.

Overall Classification Performance

Overall	Prec	Rec	F1	Supp
micro	0.821	0.821	0.821	470
macro	0.834	0.608	0.591	470
w-avg	0.834	0.821	0.821	470

Per label	Prec	Rec	F1	Supp
(ΠI)	0.582	0.768	0.662	69
(ΠE)	0.866	0.710	0.780	200
(Memb.)	0.888	0.955	0.920	200
(ΣI)	1.000	0.000	0.000	1

What these numbers tell us

- 0.821 against 0.982 *with* proof terms [Miyagawa et al. 2025]: the term is the **strongest cue**.
- (Memb.) and (ΠE) dominate backward search, and are predicted **most accurately** — where the search cost is.
- macro-F1 is held down by (ΣI) : **one** test instance.

Overall Classification Performance

Overall	Prec	Rec	F1	Supp
micro	0.821	0.821	0.821	470
macro	0.834	0.608	0.591	470
w-avg	0.834	0.821	0.821	470

Per label	Prec	Rec	F1	Supp
(ΠI)	0.582	0.768	0.662	69
(ΠE)	0.866	0.710	0.780	200
(Memb.)	0.888	0.955	0.920	200
(ΣI)	1.000	0.000	0.000	1

What these numbers tell us

- 0.821 against 0.982 *with* proof terms [Miyagawa et al. 2025]: the term is the **strongest cue**.
- (Memb.) and (ΠE) dominate backward search, and are predicted **most accurately** — where the search cost is.
- macro-F1 is held down by (ΣI) : **one** test instance.

Does the Neural Model Weaken the Prover?

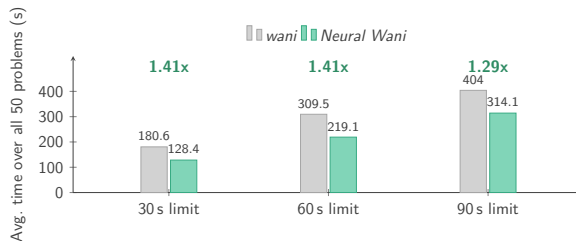
$$\begin{array}{c} \textit{wani} \\ 8 / 50 \end{array} = \begin{array}{c} \textit{Neural Wani} \\ 8 / 50 \end{array}$$

and they are **the same 8 problems** — at every time limit: 30 s, 60 s, 90 s

Every problem the baseline solves **remains solvable**
— **nothing was lost.**

Quantitative Analysis of Search Efficiency

Both provers solve **8/50** at every limit — **nothing lost**. The difference is in the time.



The limit bounds *one depth iteration*, not a whole problem.

Where the gain comes from

No gain on the **8** problems both solve; the gain is on the **42 that hit the limit**.

5

Conclusion

Neural Wani

Conclusion

Neural Wani = *wani* + **neural rule prediction**

- Predicts rules for **judgments lacking proof terms** — what proof search actually faces
- **1.41x** average speedup, **the same 8/50 solved**

Future work

- Train on **genuine proof-search data**
- Refine as more solvable problems are collected

Acknowledgements

This work was supported in part by **JST CREST** Grant Number JPMJCR2565 and **JSPS KAKENHI** Grant Number JP23H03452.

References

- Bekki, D. (2014). Representing anaphora with dependent types. *LACL*.
- Bekki, D. & Kawazoe, A. (2016). Implementing variable vectors in a CCG parser. *LACL*.
- Bekki, D. & Mineshima, K. (2017). Context-passing and underspecification in Dependent Type Semantics. *Modern Perspectives in Type-Theoretical Semantics*.
- Daido, H. & Bekki, D. (2020). Development of an automated theorem prover for the fragment of DTS. *LENLS 17*.
- Hochreiter, S. & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8).
- Kawazoe, A., Tanaka, R., Mineshima, K. & Bekki, D. (2015a). A framework for constructing multilingual inference problem sets. *MLRKRS*.
- Kawazoe, A., Tanaka, R., Mineshima, K. & Bekki, D. (2015b). An inference problem set for evaluating semantic theories and semantic processing systems for Japanese. *LENLS 12*.
- Kogkalidis, K., Melkonian, O. & Bernardy, J.-P. (2024). Learning structure-aware representations of dependent types. *NeurIPS 37*.
- Martin-Löf, P. (1984). *Intuitionistic Type Theory*. Bibliopolis.
- Miyagawa, N., Tagami, S. & Bekki, D. (2025). Toward the development of an automated theorem prover “Neural Wani” for Dependent Type Theory (in Japanese). *JSAI 39*.
- Rocktäschel, T. & Riedel, S. (2017). End-to-end differentiable proving. *NIPS 30*.
- Tomita, A., Matsubara, M., Daido, H. & Bekki, D. (2025). Natural language inference with CCG parser and automated theorem prover for DTS. *BriGap-2*.
- Wang, M., Tang, Y., Wang, J. & Deng, J. (2017). Premise selection for theorem proving by deep graph embedding. *NIPS 30*.

Related Work: Logic-based NLI & Automated Reasoning

Model-theoretic

Surface $\rightarrow$ FOL via λ -calculus; entailment / consistency decided by first-order provers + model builders [Blackburn & Bos 2005], scaled with wide-coverage CCG semantics [Bos et al. 2004].

Proof-theoretic

Entailment by symbolic inference: natural logic [MacCartney & Manning 2008], tableau [Abzianidze 2015], Coq-based [Chatzikyriakidis & Luo 2014; Mineshima et al. 2015; Martínez-Gómez et al. 2017; Chatzikyriakidis & Bernardy 2019].

Our position

Proof-theoretic branch. DTS computes entailment **purely through proof search**, with no external models. We accelerate the dedicated DTT prover *wani* by **guiding its search with a neural model**; its YES/NO/UNK mirrors the classic entailment/contradiction/unknown decision.

Difference from Previous Work

Previous work [Miyagawa et al. 2025] predicted inference rules from judgments with **known** proof terms (micro-F1: 0.982).

- Actual backward inference relies on **proof search queries** where proof terms are **unknown**.
 - Applying rules recursively generates new subgoals that also lack proof terms.
- ⇒ Practical provers must predict rules directly from these **incomplete judgments**.

This study addresses this exact task — a crucial step for constructing proof trees and enabling effective automated search in *Neural Wani*.

A Concrete Example: Toy NLI Problem

- **Premise 1:** “Subete no gakusei ga aruita” (“Every student walked”)
- **Premise 2:** “John wa gakusei de aru” (“John is a student”)
- **Hypothesis:** “John wa aruita” (“John walked”)
- **Answer: YES (entailment)** — *wani* finds a proof term by **backward inference**, using only (ΠE) and (Membership).

$$\begin{array}{c}
 \frac{}{g : (x:\text{entity}) \rightarrow (\text{student}(x) \rightarrow \text{walk}(x))} \text{(Membership)} \quad \frac{}{\text{john} : \text{entity}} \text{(Membership)} \\
 \hline
 \frac{}{g(\text{john}) : \text{student}(\text{john}) \rightarrow \text{walk}(\text{john})} \text{(\Pi E)} \quad \frac{}{s : \text{student}(\text{john})} \text{(Membership)} \\
 \hline
 g(\text{john})(s) : \text{walk}(\text{john}) \text{(\Pi E)}
 \end{array}$$

Simplified DTS representation; (Membership) leaves look up a premise or a constant.

A Fragment of DTT

wani restricts DTT to a **fragment**. Core type constructors:

- **Π -type** (dependent function): $(x:A) \rightarrow B$
- **Σ -type** (dependent pair): $\left[\begin{array}{c} x:A \\ B \end{array} \right]$
- ... and $(=)$ -type, $\top$, $\perp$, sorts type/kind

Π/Σ -types let consequent types **depend on antecedents** $\rightarrow$ handles phenomena such as **anaphora**.

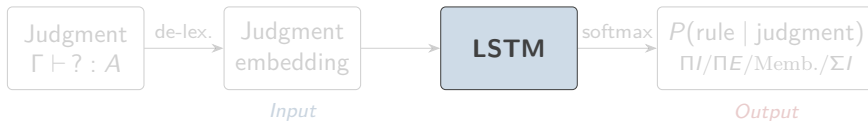
Judgments as a Data Structure (Haskell)

A DTT judgment is represented in *wani* (Haskell) as:

```
data Judgment = Judgment {  
    signature :: Signature -- [(Text, Preterm)]  
    , context  :: Context   -- [Preterm]  
    , term     :: Preterm  
    , typ      :: Preterm  
} deriving (Eq, G.Generic)
```

$\Gamma \vdash_{\sigma} M : A$: under signature σ , there exists a proof diagram from context Γ to the type assignment $M : A$.

Why a Lightweight LSTM, not a Transformer [Vaswani et al. 2017]?



- **Low latency** — the model is queried at *every* search-tree node. A slow model would consume the time we are trying to save.
- Rule prediction needs only the **local structure** of the judgment.

Tokenization & De-lexicalization

Each element of a judgment is mapped to **abstract tokens** (open vocabulary → logical structure):

- **Structural tokens** — logical constants (Π , Σ , $=$, type) and delimiters (EOCon, EOSig) each get a unique token.
- **Constant placeholders** — lexical constants → Word1..Word31 by order of appearance (overflow → UNKNOWN).
- **Variable placeholders** — bound variables → Var'0..Var'6 by De Bruijn index (overflow → Var'unknown); this **resolves α -equivalence**.

De-lexicalization makes the model **agnostic to lexical semantics** and **independent of the surface language** — it learns abstract structural templates.

From Judgment to Token Sequence

Each component of the de-lexicalized toy goal is closed by its own EO marker:

Judgment component	Token sequence
signature =	Word1 COMMA Pi' Entity' EOPre ... EOPair
[(student, entity->type),	Word2 COMMA Pi' Entity' EOPre ... EOPair
(walk, ...), (john, entity)]	Word3 COMMA Entity' EOPre EOPair EOSig
context =	App' Word1 EOPre Word3 EOPre EOPre
[student(john), (x:entity)->...]	Pi' Entity' EOPre Pi' App' Word1 EOPre Var'0 ... EOCon
term = ... <i>excluded</i>	<i>nothing is encoded — the term is unknown during proof search</i>
type = walk(john)	App' Word2 EOPre Word3 EOPre EOPre EOTyp

EOSig / EOCon / EOTyp = end of **signature** / **context** / **type**.

Embedding Methods Compared (Preliminary Study)

Four ways to serialize a judgment into a token sequence were compared: **parentheses**, **SEP**, **EO~**, and **no delimiter**.

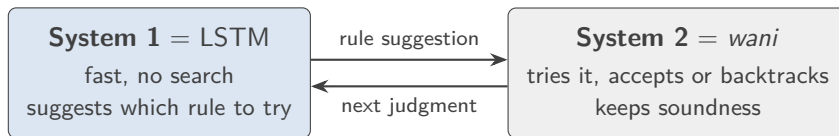
Embedding method	micro-F1
EO~ delimiters	0.982
No delimiter	0.966
Parentheses	0.963
SEP delimiters	0.953

The **EO~** method scored highest $\Rightarrow$ we adopt it.

* Preliminary study [Miyagawa et al. 2025]: rule prediction from judgments that *contain* proof terms.

Cognitive Parallelism: Intuition-Guided Search

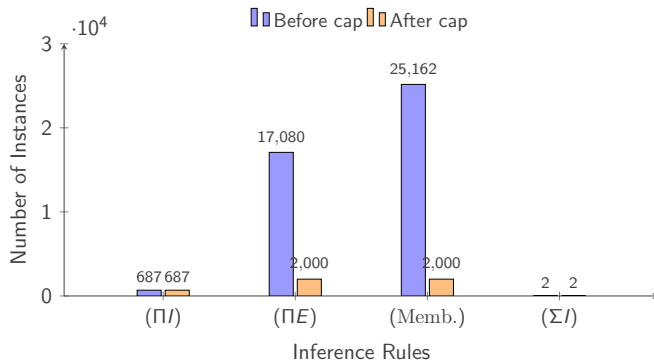
Like a logician's "logical intuition": promising rules first, not exhaustive search. An *interpretive analogy* — not a cognitive claim.



Dual Process Theory [Evans & Frankish 2009]: the pairing navigates DTT's undecidable search space more effectively than either part alone.

Label Distribution and Class Imbalance

Only four backward-inference rules actually occur; the distribution is severely imbalanced, so we **cap each rule at 2,000 instances**.



Rationale for Focusing on Backward Inference

Neural Wani aims to optimize computationally expensive proof search stages.

- **Forward** reasoning has strictly limited rules.
- **Backward** inference involves diverse candidates.
- Unrestricted rules like (ΠE) and (DNE) cause an **explosive expansion of the search space**.

Forward	Backward		
(Membership)	(ΠI)	(ΣI)	(Membership)
(ΣE)	(ΠE)	(ΣF)	(DNE)
$(=I)$	(ΠF)	$(\top I)$	(EFQ)
$(=E)$	$(=F)$		

Inference rules in *wani*, grouped by direction.

Rationale for Excluding Formation Rules

- The conclusion of a **formation rule** typically yields a judgment with type or kind as its type.
- For such judgments, the applicable rule is **syntactically and uniquely determined** (few exceptions, e.g. (CON)).

$$\begin{array}{c}
 \overline{x : A}^i \\
 \vdots \\
 \frac{A : s_1 \quad B : s_2}{(x : A) \rightarrow B : s_2} (\Pi F), i \\
 (s_1, s_2) \in \{(\text{type}, \text{type}), (\text{type}, \text{kind}), (\text{kind}, \text{kind})\}
 \end{array}$$

The application of formation rules is **deterministic** — no probabilistic prediction needed. → We exclude them from the task and evaluation.

Experimental Setup for the Neural Model

LSTM [Hochreiter and Schmidhuber 1997] hyperparameters:

Hyperparameters	
Number of Epochs	10
Learning Rate	5×10^{-4}
Number of Layers	1
Dropout	None
Input Size	128
Hidden State Size	128
BPTT Steps	32

Experimental Precautions (Fair Comparison)

To minimize bias in the *wani* vs *Neural Wani* timing:

- **Lazy evaluation:** queries fully evaluated and manual GC triggered before each test (Haskell), preventing resource interference between runs.
- **Execution order:** alternated — baseline first for half the problems, *Neural Wani* first for the other half.
- **Caching:** *Neural Wani*'s rule-prediction cache cleared between different time-limit experiments to keep measurements independent.

Why micro-F1 > macro-F1?

Overall **micro-F1** = 0.821, but **macro-F1** = 0.591.

- The gap stems from the **imbalanced label distribution**: micro-F1 is dominated by high-frequency rules, while macro-F1 weights all classes equally (minority rules remain hard).
- Lower than the prior 0.982 [Miyagawa et al. 2025] — expected, since removing the proof term (the strongest cue) makes the task substantially harder.

Why this profile still helps

Most search-tree nodes involve high-frequency rules ((Membership), (ΠE)); accurate prioritization there puts the productive rule **first** at most nodes, which is where the **time saving** comes from.

Full Comparison Table

Metric	30s Limit	60s Limit	90s Limit
Total tests	50	50	50
Normal success	8/50	8/50	8/50
Neural success	8/50	8/50	8/50
Avg. Normal time (all)	180.619 sec	309.516 sec	404.035 sec
Avg. Neural time (all)	128.376 sec	219.143 sec	314.057 sec
Speedup (all avg)	1.41x	1.41x	1.29x
Avg. Normal time (both succ.)	2.093 sec	2.068 sec	2.062 sec
Avg. Neural time (both succ.)	4.915 sec	2.087 sec	2.082 sec
Speedup (both succ. avg)	0.43x	0.99x	0.99x

The **both succ.** rows cover only the 8 problems solved by both systems. Where the search is structurally shallow, the LSTM's constant inference cost is not offset by the branches it saves.

A Solved JSeM Problem (Example)

One of the 8 problems solved by *both* systems — JSeM #63 (GeneralizedQuantifier):

- **Premise:** “Fukusuu no shain ga idou o kibou shite iru” (“Several employees want a transfer.”)
- **Hypothesis:** “Idou o kibou shite iru shain ga iru” (“There exists an employee who wants a transfer.”)
- **Answer:** **YES (entailment).**

Both systems build **identical proof trees**
33 rule applications: 3 (ΣI), 10 (ΣE), 20 (Membership).

Limitations & Future Work

- **Gap between training data and the actual task.**
 - We train on **type-checking data as a proxy**, since solved proof-search instances are still scarce.
 - This creates a **distribution shift** from proof search.
- **Refine the model with real proof-search data.**
 - As solved cases accumulate, retrain on authentic proof-search trajectories.
 - **Five concurrent projects (NLP2026*)** are widening what the *wani* ecosystem can solve:

Tomita & Bekki

Saeki et al.

Sakuma et al.

Kobayashi et al.

Matsubara et al.

LLM-based knowledge completion for *lightblue*

a Yesod development environment for *lightblue*

Japanese compound verbs in DTS

Neural DTS: hyperbolic classifiers

validity verification on financial texts

* NLP2026 = the 32nd Annual Meeting of the Association for Natural Language Processing (in Japanese).